

DOJO

CodeDojo

Application Desktop d'Entraînement au Code par IA

Python · PySide6 · Groq AI · MySQL Community Server

“La discipline forge le talent. Le code est notre tatami.”

Auteure **Jihane Benzerkane**

Établissement **ENSAH – Al Hoceima**

Filière **IA & Transformation Digitale**

Année **2025–2026**

Version **MVP v1.0 – Révisé**

Python 3.11

PySide6

MySQL

Groq AI

LLaMA 3.3

Table des matières

1	Cahier des Charges	2
1.1	Contexte et Problématique	2
1.2	Objectifs du Projet	2
1.3	Définition du MVP	2
1.4	Périmètre du MVP	3
1.4.1	Fonctionnalités incluses	3
1.4.2	Fonctionnalités hors périmètre (v2)	4
1.5	Acteurs du Système	4
1.6	Besoins Non Fonctionnels	5
1.7	Contraintes Techniques	5
2	Conception – Diagrammes UML	6
2.1	Diagramme des Cas d’Utilisation	6
2.2	Diagramme de Classes	7
2.3	Diagramme de Séquence – Authentification	8
2.4	Diagramme de Séquence – Soumission de Code	8
3	Architecture Technique	8
3.1	Structure du Projet	9
3.2	Schéma MySQL	10
3.3	Script SQL d’Initialisation	10
4	Plan de Développement	11
4.1	Semaine 1 – Infrastructure	11
4.2	Semaine 2 – Fonctionnalités	12
4.3	Diagramme de Gantt	12
5	Identité Visuelle et UX	12
5.1	Palette de Couleurs	13
5.2	Composants UI	13
6	Plan de Test et Qualité	13
6.1	Matrice de Tests	14
6.2	System Prompt Sensei	14
7	Livrables et Valeur Portfolio	15
7.1	Requirements	15
7.2	Livrables Finaux	15
7.3	Valeur Portfolio	16
7.4	Récapitulatif des Corrections Apportées	16

Cahier des Charges

1.1 Contexte et Problématique

Les étudiants en informatique manquent d'un environnement d'entraînement qui les pousse à réfléchir plutôt qu'à copier. Les plateformes existantes comme HackerRank ou LeetCode proposent un feedback binaire – correct / incorrect – sans accompagnement pédagogique.

Problème central identifié

Un étudiant soumis à un problème algorithmique tend à chercher la solution plutôt qu'à développer le raisonnement qui y mène. Les outils disponibles facilitent la copie plutôt que l'apprentissage profond.

CodeDojo répond à ce problème via un tuteur IA socratique : une intelligence artificielle qui ne donne jamais la réponse directement, mais guide l'apprenant ligne par ligne dans un environnement desktop immersif.

1.2 Objectifs du Projet

- ▷ Fournir un environnement d'entraînement au code Java gamifié et motivant
- ▷ Intégrer un tuteur IA (Groq / LLaMA 3.3) adoptant une pédagogie socratique
- ▷ Persister les données utilisateurs dans **MySQL** (Community Server)
- ▷ Proposer une interface desktop avec thème japonais
- ▷ Constituer un projet portfolio pour les candidatures Oracle Internship Program

1.3 Définition du MVP

Qu'est-ce que le MVP ?

MVP = **Minimum Viable Product** (Produit Minimum Viable).

C'est la version la plus simple possible d'un projet qui fonctionne réellement – juste assez de fonctionnalités pour être utilisable et démontrer la valeur, sans surcharge inutile.

Principe : livrer d'abord le cœur fonctionnel (auth + challenges + IA), puis enrichir en v1.1 et v2. Chaque version doit être utilisable seule.

Version	Contenu	Délai estimé
MVP v1.0	Auth + Challenges + IA Groq + Points	14 jours (3h/jour)
v1.1	Thème japonais poussé + animations feuilles	+1 semaine
v2.0	Spotify, mode offline, multi- langages, podium	+1 mois

TABLE 1.1 – Roadmap des versions – CodeDojo

1.4 Périmètre du MVP

1.4.1 Fonctionnalités incluses

ID	Fonctionnalité	Priorité
BF01	Inscription avec username et mot de passe (bcrypt)	Must
BF02	Connexion et vérification d'identité	Must
BF03	Affichage des challenges Java par difficulté	Must
BF04	Éditeur de code intégré (QPlainTextEdit)	Must
BF05	Validation du code avant envoi à l'IA	Must
BF06	Soumission du code au tuteur IA Groq	Must
BF07	Réponse IA socratique (hints, jamais la solution)	Must
BF08	Système de points et de niveaux en MySQL	Must
BF09	Timer Pomodoro 25 min (QTimer)	Must
BF10	Historique des sessions dans MySQL	Should
BF11	Thème japonais basique via QSS	Should

TABLE 1.2 – Besoins fonctionnels – périmètre MVP v1.0

Fonctionnalités déplacées en v1.1 (corrections vs version initiale)

Les fonctionnalités suivantes ont été retirées du MVP pour des raisons réalistes :

Spotify (QWebEngineView) : dépendance lourde et instable, zéro valeur fonctionnelle pour le cœur du projet.

Animations feuilles tombantes : effort graphique pouvant bloquer plusieurs heures

pour un résultat non critique.
Ces deux éléments seront implémentés en v1.1 une fois le MVP stable.

1.4.2 Fonctionnalités hors périmètre (v2)

Fonctionnalité	Raison du report
Lecteur Spotify	Dépendance QWebView instable
Animations feuilles	Effort graphique non critique pour le MVP
Mode offline (upload exercices)	Complexité gestion fichiers Java
Niveau Senior	Priorité Junior pour démontrer le concept
Animations podium / trophées	Effort graphique non prioritaire
Simulation de pro- jet	Feature majeure – v2 complète
Multi-langages	Java seul suffit pour le MVP

TABLE 1.3 – Fonctionnalités reportées à v1.1 et v2

1.5 Acteurs du Système

Acteur	Rôle
Utilisateur	S'authentifie, choisit un challenge, écrit du code Java, interagit avec le tuteur IA
IA (Groq)	Génère des hints socratiques, analyse le code soumis, valide les solutions
MySQL	Persiste utilisateurs, challenges, sessions et messages IA

TABLE 1.4 – Acteurs et rôles

1.6 Besoins Non Fonctionnels

ID	Catégorie	Exigence
BNF01	Performance	Temps de réponse IA < 5 secondes (Groq LPU rapide)
BNF02	Sécurité	Mots de passe jamais stockés en clair – hashage bcrypt
BNF03	Sécurité SQL	Toutes les requêtes MySQL via requêtes paramétrées (anti-injection)
BNF04	Portabilité	Application autonome, aucun navigateur requis
BNF05	Utilisabilité	Interface utilisable sans formation préalable
BNF06	Compatibilité	Fonctionne sur Windows 10+ et Linux Ubuntu 20+
BNF07	Disponibilité	Fonctionne sans internet sauf appels Groq API
BNF08	Robustesse	Gestion des erreurs : Groq timeout, MySQL déconnecté, code vide soumis

TABLE 1.5 – Besoins non fonctionnels

Note BNF07 – Clarification offline

MySQL Community Server tourne **en local** sur la machine – il ne nécessite pas d'internet. Seuls les appels à l'API Groq nécessitent une connexion internet. L'application fonctionne donc offline pour tout sauf le tuteur IA.

1.7 Contraintes Techniques**Stack technique imposée**

- **Langage** : Python 3.11+
- **UI Framework** : PySide6 (Qt6) – obligation universitaire ENSAH
- **IA** : Groq API, modèle `llama-3.3-70b-versatile` (free tier, sans carte bancaire)
- **Base de données** : MySQL Community Server + `mysql-connector-python`
- **Sécurité** : `bcrypt` pour le hashage des mots de passe
- **Configuration** : `python-dotenv` – aucune clé API en dur dans le code

Groq API – Free Tier (vérifié mai 2026)

- ✓ Inscription sur `console.groq.com` – aucune carte bancaire requise
- ✓ Limites : 30 requêtes/minute, 6 000 tokens/minute, 1 000 requêtes/jour
- ✓ Largement suffisant pour le développement et les démos portfolio
- ✓ Si limite dépassée : code HTTP 429 – à gérer avec un message d'erreur propre

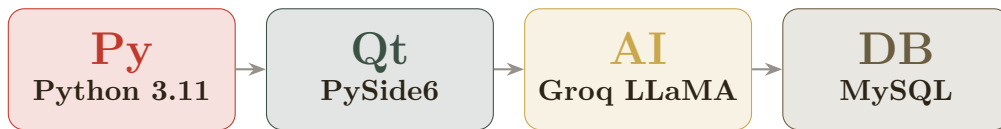


FIGURE 1.1 – Architecture technologique – CodeDojo MVP

Chapitre 2

Conception – Diagrammes UML

2.1 Diagramme des Cas d'Utilisation

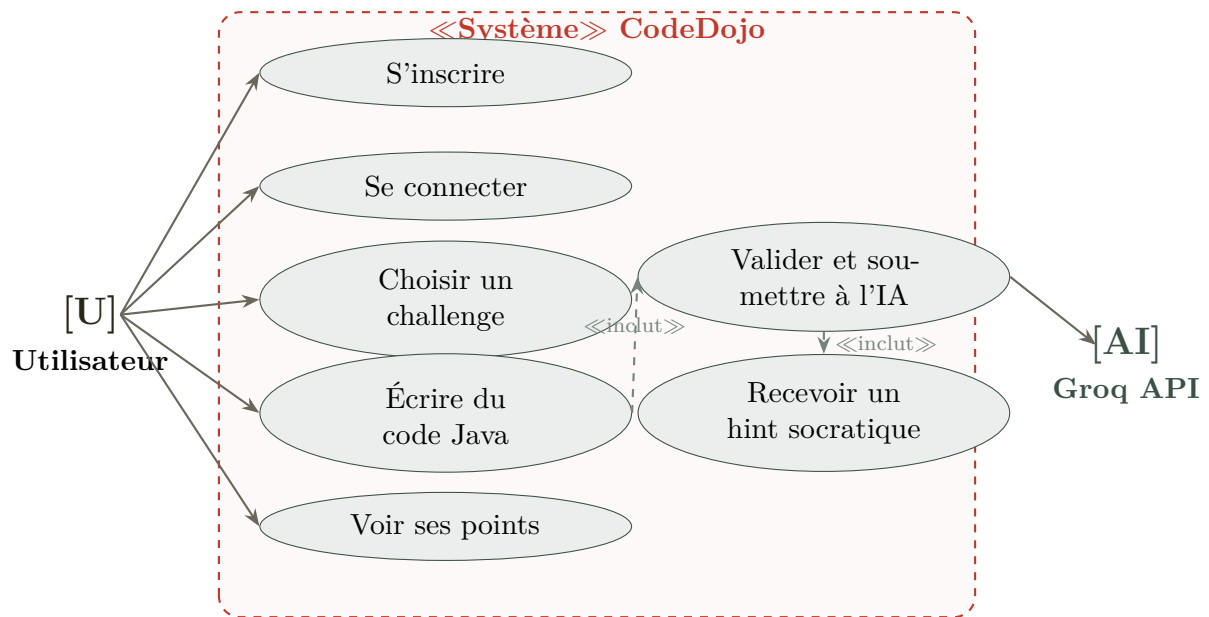


FIGURE 2.1 – Diagramme des cas d'utilisation – CodeDojo MVP

2.2 Diagramme de Classes

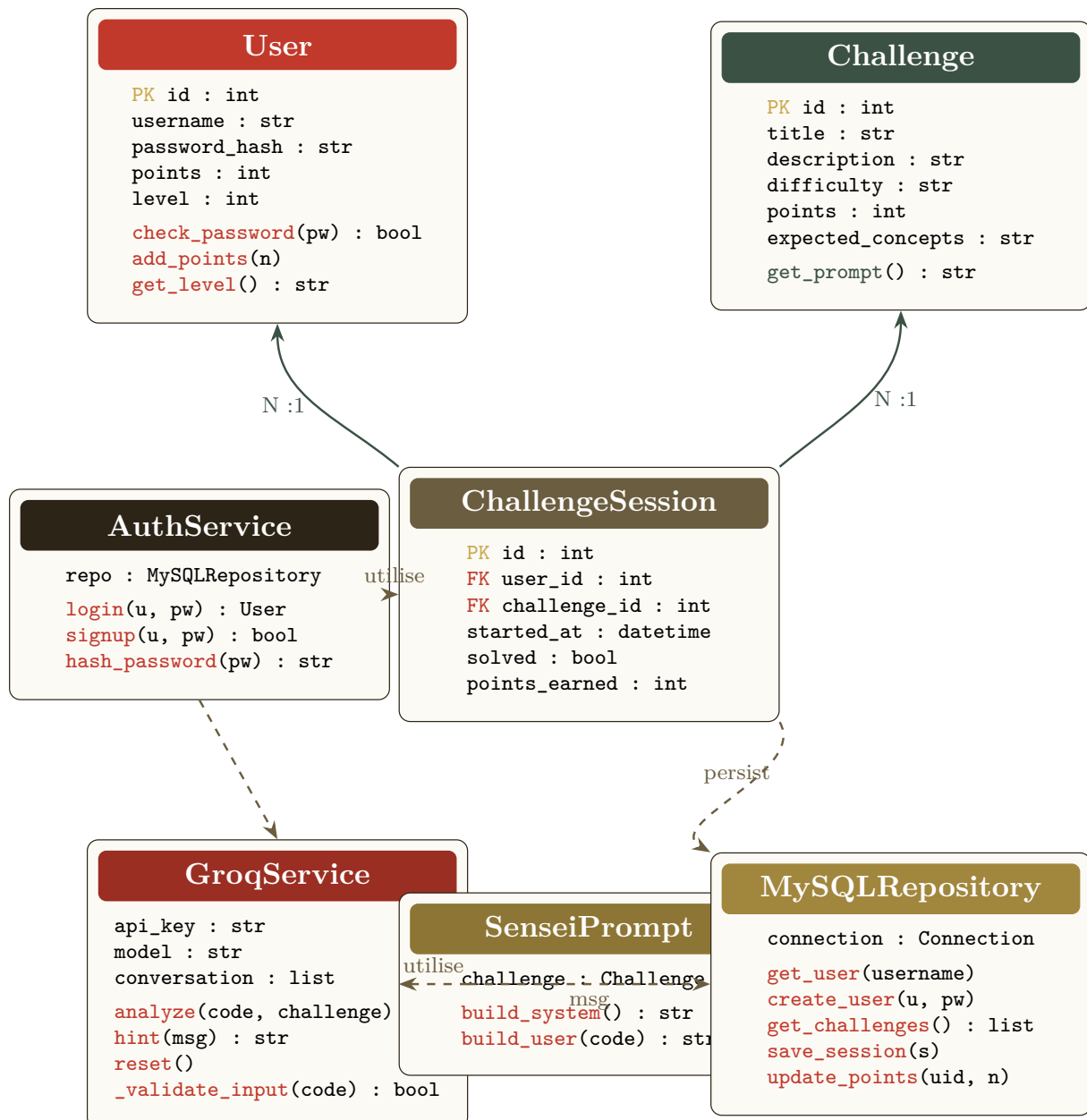


FIGURE 2.2 – Diagramme de classes – architecture logicielle CodeDojo (révisé)

Ajout vs version initiale : classe SenseiPrompt

La classe **SenseiPrompt** sépare la logique de construction des prompts du client HTTP Groq. **GroqService** gère uniquement les appels API ; **SenseiPrompt** gère le prompt engineering. Architecture plus propre, plus testable, plus lisible sur un CV.

2.3 Diagramme de Séquence – Authentification

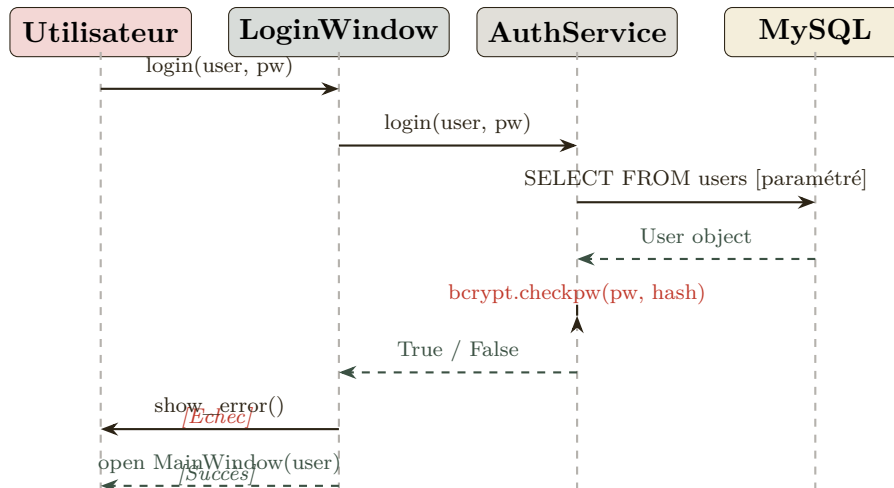


FIGURE 2.3 – Diagramme de séquence – flux d’authentification

2.4 Diagramme de Séquence – Soumission de Code

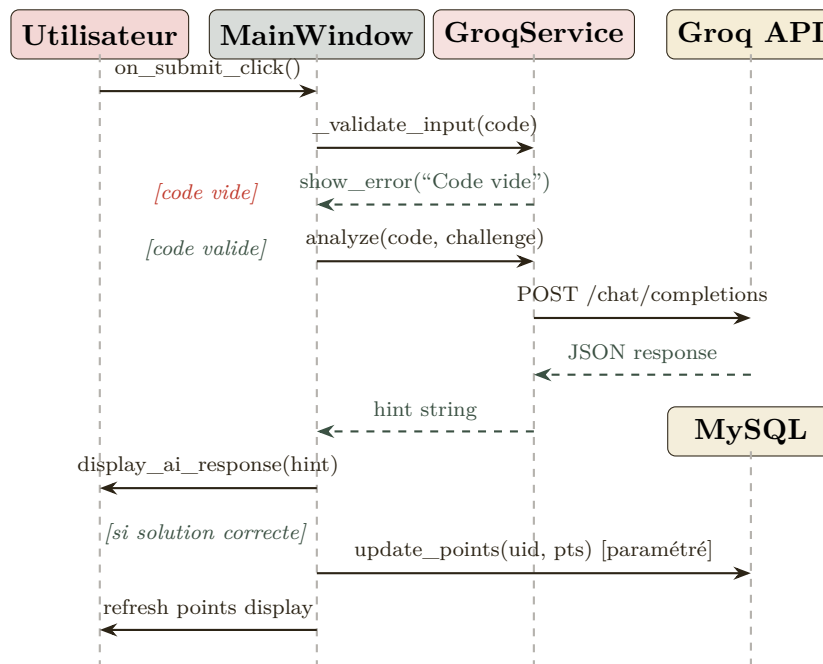


FIGURE 2.4 – Diagramme de séquence – soumission de code (avec validation)

Chapitre 3

Architecture Technique

3.1 Structure du Projet

```
codedojo/
|-- main.py
|-- ui/
|   |-- login_window.py
|   |-- main_window.py
|   |-- challenge_panel.py
|   |-- editor_panel.py
|   |-- ai_panel.py
|   |-- status_bar.py
|   '-- leaf_overlay.py           # v1.1
|-- services/
|   |-- auth_service.py
|   |-- groq_service.py          # client HTTP Groq uniquement
|   '-- sensei_prompt.py         # prompt engineering (ajout v1.0)
|-- repository/
|   '-- mysql_repo.py            # TOUJOURS requetes parametrees
|-- models/
|   '-- entities.py
|-- db/
|   |-- init_db.sql
|   '-- seed_challenges.sql
|-- assets/
|   '-- style.qss
|-- .env                         # cle GROQ_API_KEY (jamais en dur)
|-- requirements.txt
'-- README.md
```

3.2 Schéma MySQL

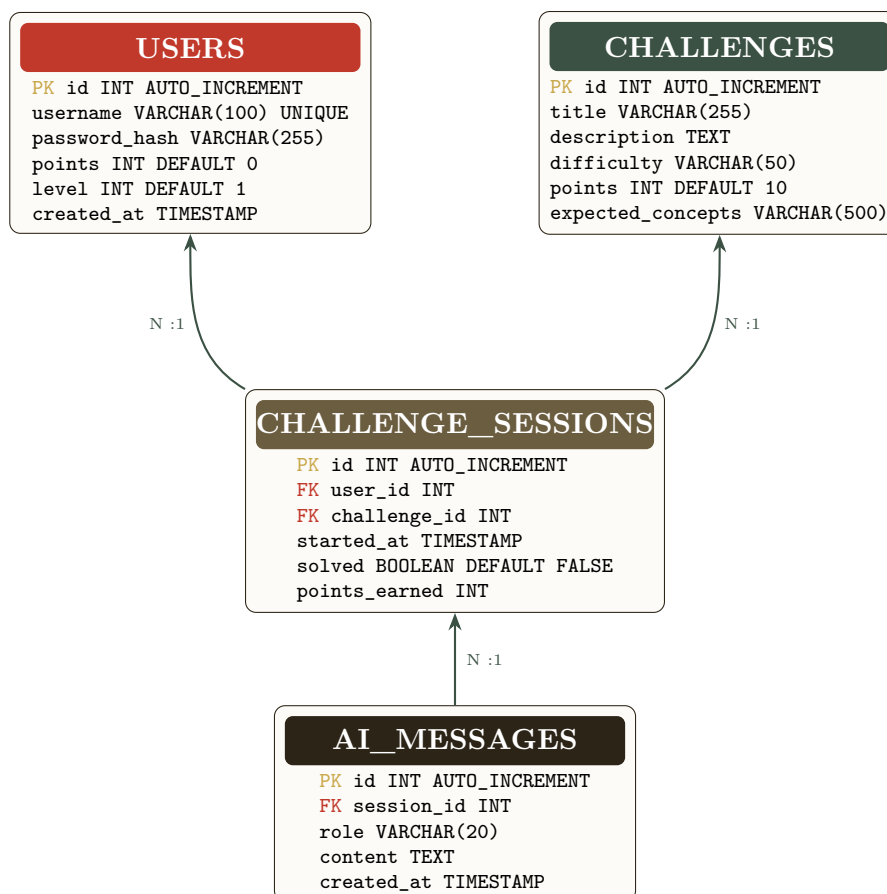


FIGURE 3.1 – Schéma relationnel MySQL – CodeDojo

3.3 Script SQL d'Initialisation

Listing 3.1 – init_db.sql – MySQL

```

1  -- =====
2  -- Database: CodeDojo
3  -- Toutes les requetes applicatives doivent etre PARAMETREES
4  -- =====
5
6  CREATE DATABASE IF NOT EXISTS codedojo CHARACTER SET utf8mb4;
7  USE codedojo;
8
9  CREATE TABLE users (
10     id            INT AUTO_INCREMENT PRIMARY KEY,
11     username      VARCHAR(100) UNIQUE NOT NULL,
12     password_hash VARCHAR(255) NOT NULL,
13     points        INT DEFAULT 0,
14     level         INT DEFAULT 1,
15     created_at    TIMESTAMP DEFAULT CURRENT_TIMESTAMP
16 );
17
18 CREATE TABLE challenges (
19     id            INT AUTO_INCREMENT PRIMARY KEY,
20     title         VARCHAR(255) NOT NULL,
21     description   TEXT NOT NULL,
22     difficulty    VARCHAR(50) DEFAULT 'junior',
23     points        INT DEFAULT 10,
24     language      VARCHAR(50) DEFAULT 'java',

```

```

25     expected_concepts VARCHAR(500)
26 );
27
28 CREATE TABLE challenge_sessions (
29     id INT AUTO_INCREMENT PRIMARY KEY,
30     user_id INT,
31     challenge_id INT,
32     started_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
33     solved BOOLEAN DEFAULT FALSE,
34     points_earned INT DEFAULT 0,
35     FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE,
36     FOREIGN KEY (challenge_id) REFERENCES challenges(id) ON DELETE
        CASCADE
37 );
38
39 CREATE TABLE ai_messages (
40     id INT AUTO_INCREMENT PRIMARY KEY,
41     session_id INT,
42     role VARCHAR(20),
43     content TEXT,
44     created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
45     FOREIGN KEY (session_id) REFERENCES challenge_sessions(id)
46         ON DELETE CASCADE
47 );

```

Chapitre 4

Plan de Développement

4.1 Semaine 1 – Infrastructure

Jour	Objectif	Livrable	Effort
J1	Setup complet	venv, packages, MySQL, .env	Faible
J2	MySQL schema + seed	Tables + 15 challenges Java	Faible
J3	MySQLRepository	Connexion + CRUD testé en terminal	Moyenne
J4	AuthService	login() signup() bcrypt testés	Moyenne
J5	LoginWindow Py-Side6	Fenêtre fonctionnelle connectée DB	Haute

TABLE 4.1 – Semaine 1 – Infrastructure

Note planning – J3 révisé vs version initiale

La version initiale regroupait MySQLRepo + AuthService sur J3, ce qui était irréaliste pour une débutante PySide6. La version révisée décompose : J3 = MySQL seul, J4 = Auth, J5 = UI Login. Cela donne le temps de gérer les erreurs de connexion MySQL (host down, mauvais credentials) proprement.

4.2 Semaine 2 – Fonctionnalités

Jour	Objectif	Livrable	Effort
J6	MainWindow squelette	3 panneaux, pas de crash	Haute
J7	ChallengePanel + EditorPanel	Liste MySQL + zone code	Haute
J8	SenseiPrompt + GroqService	Prompts construits et testés	Moyenne
J9	Intégration IA complète	Submit → réponse dans UI	Haute
J10	Points + Timer QTimer	Points MySQL + 25min Pomodoro	Moyenne
J11	QSS thème japonais basique	Style appliqué, couleurs cohérentes	Moyenne
J12	Tests + bugfix	Crashes corrigés, edge cases gérés	Haute
J13	README + démo	Screen recording 3-5 min	Faible
J14	Buffer	Polish final	—

TABLE 4.2 – Semaine 2 – Fonctionnalités

4.3 Diagramme de Gantt

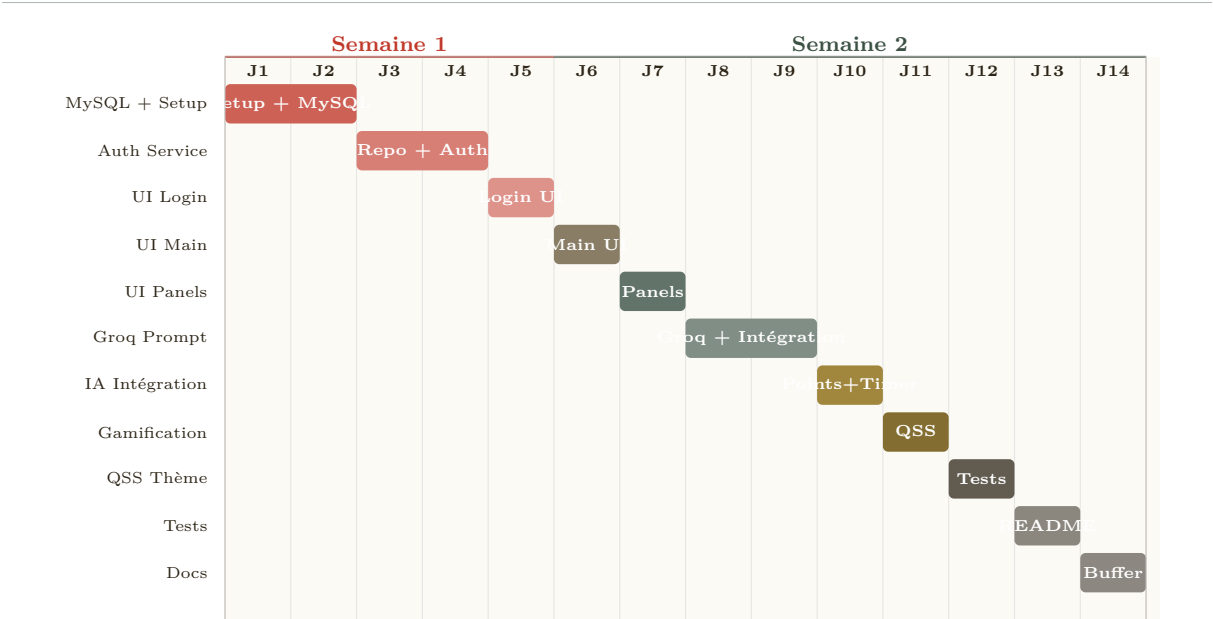


FIGURE 4.1 – Planning de développement – 14 jours (révisé)

Chapitre 5

Identité Visuelle et UX

5.1 Palette de Couleurs

Aperçu	Nom	Hex	Usage
	Charbon	#2C2416	Texte principal, headers
	Cramoisi	#C1392B	Accents, boutons CTA, titres
	Vert forêt	#3B5143	Sections, icônes nature
	Bois	#6B5D3F	Textes secondaires, bordures
	Parchemin	#F5EDD6	Fond des cartes, backgrounds
	Or	#C9A84C	Badges, points, trophées

TABLE 5.1 – Palette de couleurs – CodeDojo Japanese Dojo Theme

5.2 Composants UI

Composants UI – MVP v1.0

- ▷ **LoginWindow** : formulaire username/password, boutons Sign In / Sign Up
- ▷ **MainWindow** : 3 panneaux – ChallengePanel (gauche), EditorPanel (centre), AIPanel (droite)
- ▷ **StatusBar** : points actuels, niveau, timer Pomodoro 25 :00
- ▷ **ChallengePanel** : liste QListWidget filtrée par difficulté (Junior)
- ▷ **EditorPanel** : QPlainTextEdit avec coloration syntaxique basique + bouton Submit
- ▷ **AIPanel** : zone de conversation scrollable avec les hints du Sensei

Composants UI – v1.1 (post-MVP)

- ★ **Feuilles tombantes** : overlay transparent, QTimer, effet visuel ambiance
- ★ **Lumière solaire** : gradient d'opacité se déplaçant lentement
- ★ **Cartes glassmorphism** : rgba(255,255,255,0.55) + ombre douce QSS
- ★ **Lecteur Spotify** : QWebEngineView + iframe Spotify Embed

Chapitre 6

Plan de Test et Qualité

6.1 Matrice de Tests

Test	Type	Outil	Critère
Signup crée un user en MySQL	Intégration	Manuel	SELECT confirme
Login mauvais pass-word	Unitaire	pytest	retourne None
Password jamais en clair	Sécurité	SQL	hash bcrypt en DB
Requêtes SQL paramétrées	Sécurité	Code review	aucun f-string SQL
Groq répond	Intégration	pytest	réponse < 5s
IA ne donne pas la solution	Manuel	5 tests	0 solution directe
Code vide soumis	Edge case	Manuel	erreur propre affichée
Points mis à jour	Intégration	SQL	points incrémenté
Timer fonctionne	Manuel	observation	25 :00 → 00 :00
MySQL coupé	Edge case	réseau	dialogue d'erreur
Groq 429 (rate limit)	Edge case	mock	message lisible

TABLE 6.1 – Plan de test complet – CodeDojo MVP (révisé)

6.2 System Prompt Sensei

Listing 6.1 – sensei_prompt.py – SenseiPrompt

```

1 # sensei_prompt.py
2 # Separ\{e} de groq_service.py pour meilleure testabilit\{e}
3
4 class SenseiPrompt:
5     SYSTEM_TEMPLATE = """
6 You are Sensei, a strict but fair coding mentor inside CodeDojo.
7
8 Rules -- never break them:
9 - NEVER give the complete solution. Not even partially.
10 - Ask ONE focused question that makes the student think.
11 - Point to the LINE or CONCEPT, never the fix.
12 - Under 6 sentences per response.
13 - Tone: calm, direct, like a judo sensei.
14 - Always end with a question or next step.
15 - If code is empty or random text: ask the student
16   to write real code before you can help.
17
18 Challenge: {challenge_title}
19 Description: {challenge_description}
20 Expected concepts: {expected_concepts}
21 """
22
23     def __init__(self, challenge):
24         self.challenge = challenge
25

```

```

26     def build_system(self) -> str:
27         return self.SYSTEM_TEMPLATE.format(
28             challenge_title=self.challenge.title,
29             challenge_description=self.challenge.description,
30             expected_concepts=self.challenge.expected_concepts
31         )
32
33     def build_user(self, code: str) -> str:
34         if not code or not code.strip():
35             return "[NO CODE SUBMITTED]"
36         return f"Here is my current code:\n\n{code}"

```

Chapitre 7

Livrables et Valeur Portfolio

7.1 Requirements

```

PySide6==6.7.0
groq==0.9.0
mysql-connector-python==8.4.0
bcrypt==4.1.3
python-dotenv==1.0.1
pytest==8.2.0

```

7.2 Livrables Finaux

Livrable	Contenu
GitHub Repository	Code source complet, .gitignore, README
Dossier technique PDF	CDC, UML, architecture (ce document)
Démo vidéo	Screen recording 3-5 min
seed_challenges.sql	15 challenges Java commentés
Rapport de tests	pytest + cas manuels

TABLE 7.1 – Livrables attendus

7.3 Valeur Portfolio

Pourquoi CodeDojo est un projet portfolio remarquable

- ★ **MySQL** : base de données open-source standard de l'industrie
- ★ **Groq + LLaMA 3** : compétence GenAI concrète sur projet original
- ★ **PySide6** : framework desktop professionnel, différenciant sur CV
- ★ **Architecture propre** : UI / Service / Repository – pattern entreprise
- ★ **Sécurité prouvée** : bcrypt + requêtes paramétrées anti-injection SQL
- ★ **Impact social** : aide réelle aux étudiants – narrative forte en entretien
- ★ **Compétences démontrées** : Python, SQL, API REST, UI/UX, Tests

7.4 Récapitulatif des Corrections Apportées

Correction	Détail
Groq Free Tier documenté	Limites exactes 2026 : 30 req/min, 1000/jour, sans CB
Spotify retiré du MVP	Dépendance instable – déplacé en v1.1
Animations retirées du MVP	Effort graphique non critique – déplacé en v1.1
Planning J3 révisé	MySQL seul J3, Auth J4, Login UI J5
SenseiPrompt séparé	Nouveau fichier sensei_prompt.py séparé de groq_service.py
Validation code ajoutée	_validate_input() avant tout appel Groq
BNF03 anti-injection SQL	Requêtes paramétrées explicitement dans les exigences
BNF07 clarification offline	MySQL = local, seul Groq nécessite internet

TABLE 7.2 – Journal des corrections – v1.0 révisée

CodeDojo

“La voie du code commence par une seule ligne.”

Jihane Benzerkane · ENSAH Al Hoceima · 2025–2026

Python · PySide6 · Groq AI · MySQL